

Implementasi *IoT Data Storage* Dengan Menggunakan Sistem Basis Data Terdistribusi Berbasis *MySQL Cluster*

Bela Liska Hartawati

Program Studi Teknik Informatika, Fakultas Ilmu Komputer, Universitas Brawijaya
Email: liskabella@gmail.com

Abstrak

Internet of Things (IoT) adalah teknologi jaringan yang memungkinkan beberapa perangkat dengan koneksi internet dapat terhubung satu dan yang lainnya. Perangkat yang saling terhubung dapat memudahkan pertukaran data yang kemudian disimpan dalam media penyimpanan tertentu. Permasalahan yang muncul yaitu perangkat *IoT* memiliki penyimpanan yang terbatas serta kemampuan komputasi yang berbeda antar perangkat. Solusi dari permasalahan tersebut yaitu menggunakan sistem penyimpanan dan komputasi berbasis *cloud storage* yang dapat menyediakan penyimpanan dan komputasi yang tidak terbatas. *Cloud storage* untuk *IoT* memerlukan sistem manajemen basis data SQL yaitu *MySQL cluster*. *MySQL cluster* dapat menyediakan *backup data* dengan baik dan *high availability*. Pengujian dilakukan dengan skenario perbedaan jumlah *request per second (rps)*. Pengujian kinerja diuji dengan menggunakan parameter *throughput* dan *latency*. Hasil pengujian *throughput* menunjukkan nilai *throughput* tertinggi yang didapatkan untuk HTTP GET yaitu 86,20 Kb/s pada 200 rps sedangkan untuk HTTP POST memiliki nilai tertinggi sebesar 115,10 Kb/s pada 250 rps. Ketika terjadi peningkatan rps maka nilai *throughput* juga meningkat, tetapi ketika kecepatan jaringan internet tidak stabil mempengaruhi besar kecilnya nilai *throughput*. Sedangkan *latency* tercepat yaitu 89,28 ms pada 50 rps. Semakin banyak *request* maka *server* memerlukan waktu yang lebih lama sehingga dapat disimpulkan semakin tinggi rps maka semakin tinggi nilai *latency*.

Kata kunci: *Internet of Things, IoT Data Storage, Basis Data, MySQL Cluster*

Abstract

Internet of Things (IoT) is a network technology that allow multiple devices which has an internet connection to connect with other devices. Devices which connect to each other are possibly to exchange of data, then it will save in device storage. There are problems with IoT device such as limited storage between IoT devices. These problem can be solved by using cloud storage with computing systems that have been integrated with IoT devices. Cloud storage are provides unlimited storage, unlimited computing and it can be accessed from anywhere and anytime. Cloud storage for IoT requires a SQL database namely MySQL Cluster. MySQL cluster are provides good data backup and high availability system when one server is down. A server down in MySQL Cluster wasn't a problem because the other server will replaces server down performance so the whole system will always be available. Performance test scenario used different request per second (rps). Performance test are using two kind of parameter they are throughput and latency. Throughput test showed the highest throughput for HTTP GET is 86.20 Kb/s at 200 rps while HTTP POST got highest throughput 115,10 Kb/s at 250 rps. If rps increased, throughput would increase but also it affected by internet connection. Unstable internet connection determined high or low of throughput had shown in HTTP GET 250 rps. Latency test showed that the fastest latency is 89.28 ms at 50 rps. Rps increased make time of latency become longer. The conclusion of latency is increasing of request will increase time of latency.

Keywords: *Internet of Things, IoT Data Storage, Database, MySQL Cluster*

1. PENDAHULUAN

Internet of Things (IoT) adalah suatu teknologi jaringan yang dapat menghubungkan

beberapa perangkat keras dengan berbagai protokol yang berbeda (Guoqiang dkk,2013). *IoT* muncul sebagai teknologi baru yang digunakan untuk mengekspresikan jaringan

telekomunikasi nirkabel modern yang saling terhubung dalam jaringan infrastruktur global yang dinamis serta berupaya menerapkan konsep konektivitas apapun, dimana saja dan kapan saja (Ali, 2015). Tujuan utama dari *IoT* adalah memungkinkan adanya komunikasi dan berbagi perangkat antara pengguna satu sama lain, sehingga dapat memberikan informasi ataupun melakukan pertukaran data yang ditangkap oleh perangkat sensor dan kemudian disimpan dalam sebuah media penyimpanan untuk dilakukan proses komputasi.

Masalah yang muncul pada *IoT*, yaitu perangkat *IoT* memiliki kendala keterbatasan dalam hal penyimpanan dan kemampuan komputasi dikarenakan volume data yang dihasilkan sulit untuk disimpan dan diproses pada *platform* lokal akibat pendistribusianya secara luas (Pratama, 2018). Hal tersebut menciptakan kekhawatiran mengenai kinerja dan kehandalan pada sistem yang akan dibangun. Kinerja merupakan hasil kerja yang dapat digunakan untuk mencapai tujuan dibuatnya suatu sistem sedangkan kehandalan adalah kemampuan sistem dalam memberikan solusi yang akurat dan terpercaya. Permasalahan tersebut dapat diatasi dengan melihat kapasitas penyimpanan dan kemampuan komputasi yang lebih baik. Solusi tersebut adalah sistem komputasi berbasis *cloud* yang nantinya akan diintegrasikan dengan perangkat *IoT*.

Cloud Data Storage merupakan bagian dari *cloud computing*. Pada penelitian Zhang, dkk (2010) menyebutkan jika *cloud* merupakan kumpulan dari beberapa *computing resources* yang bisa dikonfigurasi dan dapat diakses darimana saja, selain itu *resources*-nya bisa dikurangi atau ditambah dengan cepat dan mudah. Pada penelitian tersebut juga menyebutkan jika *cloud* secara virtual mempunyai karakteristik yang tidak terbatas dalam penyimpanan dan komputasi. *Cloud* memiliki layanan yang mudah digunakan dan mempunyai sifat *high availability*. *Cloud data storage* memiliki fungsi yang tidak dimiliki oleh media *sharing* lainnya yaitu tidak perlu mengunggah data yang ingin dibagikan pada saat melakukan proses transfer data (Rai & Patil, 2017).

Berdasarkan pada permasalahan diatas maka diperlukan sistem penyimpanan data *IoT* berupa sistem manajemen basis data SQL yaitu

MySQL Cluster. Pada penelitian Andhikha (2018) menunjukkan bahwa *MySQL cluster* memiliki kinerja yang baik. Kinerja tersebut ditinjau dari keberhasilannya dalam pengujian *backup data* dan *high availability*. Hal tersebut ditunjukkan dengan jumlah data yang berhasil dibuat serta akurasi data pada setiap *server*. Selain pengujian tersebut, terdapat pengujian *failure* yang menjadi salah satu keunggulan *MySQL cluster* yaitu dapat menggantikan salah satu *server database* yang mati agar sistem masih dapat berjalan.

Berdasarkan penjelasan diatas *MySQL cluster* digunakan sebagai sistem penyimpanan *data storage*. *IoT* dalam penelitian ini terdiri dari beberapa komponen, yaitu *perception layer* yang merupakan lapisan *IoT* paling bawah semacam komponen pembentuk *IoT* dan pengumpul data yang meliputi sensor berupa laptop atau hp, *gateway*, *cloud interface*, *cloud data storage*, *master DB*, dan *slave DB*. Penelitian dengan menggunakan *MySQL cluster* dapat memberikan performa yang lebih cepat untuk melakukan *backup data* secara *realtime*, memungkinkan adanya replikasi atau *clustering database* ketika terjadi kegagalan dan juga mempunyai spesifikasi yang digunakan dalam menyimpan data yang sangat besar. Dengan demikian, fokus dari penelitian ini adalah untuk membangun sistem penyimpanan data dalam *IoT* dengan menggunakan *cloud storage* dan sistem manajemen basis data SQL dengan *MySQL cluster*. Parameter uji yang digunakan untuk mengetahui kinerja *MySQL cluster* yaitu parameter throughput dan latency. Throughput merupakan kecepatan pengiriman data mulai paket dikirim hingga paket diterima sedangkan latency adalah waktu yang dibutuhkan ketika paket dikirim hingga paket diterima. Penggunaan parameter tersebut untuk *MySQL Cluster* dikarenakan kinerja dari sistem *clustering* diukur menggunakan kecepatan paket dikirim dan waktu yang dibutuhkan saat pengiriman paket antar cluster hingga informasi sampai ke pengguna.

2. TINJAUAN PUSTAKA

Tinjauan pustaka berisi beberapa pustaka dari penelitian sebelumnya yang berkaitan dengan penelitian yang akan dilakukan. Pustaka penelitian yang digunakan yaitu penelitian yang berkaitan dengan penggunaan *data storage* sebagai penyimpanan untuk perangkat *IoT*.

Penelitian yang dilakukan oleh I. Andhikha (2018) yang berjudul “Implementasi dan Analisis Kinerja MySQL Cluster Menggunakan Metode *Load Balancing*”. Penelitian tersebut mengangkat permasalahan *request* yang terlalu banyak mengakibatkan layanan tidak tersedia serta penurunan kinerja sistem. Penelitian ini mengusulkan penerapan MySQL cluster sebagai database pada sistem *load balancing*. Penggunaan MySQL cluster bertujuan untuk menyediakan layanan agar tetap tersedia tanpa mengalami kegagalan ketika terjadi *request* yang tinggi dengan menggunakan beberapa server dalam sistem. *Load balancing* digunakan sebagai pembagi beban agar beban tidak menumpuk pada salah satu server saja yang dapat mengakibatkan *overload* hingga menurunnya kinerja layanan.

Pada penelitian Andhikha menunjukkan jika penggunaan MySQL cluster dengan metode *load balancing* dapat berjalan dengan baik. Hal tersebut dilihat dari pembagian beban pada setiap server terbagi secara merata yang ditunjukkan dengan penggunaan CPU dan memory terpakai sama rata pada setiap server, sedangkan dengan metode *default* tidak merata. MySQL cluster memiliki kinerja yang baik pada pengujian *high availability*, *backup data* pada kedua server memiliki data yang sama. Pada pengujian *failure*, MySQL cluster dapat menjalankan mekanisme *failover* dimana dapat menggantikan tugas server database yang mati agar layanan tetap tersedia. Berdasarkan penelitian tersebut, penelitian yang akan dilakukan tetap menggunakan MySQL cluster sebagai database. Penggunaan MySQL cluster pada penelitian ini bertujuan untuk mengatasi keterbatasan storage pada perangkat IoT. Selain itu MySQL cluster dapat menjalankan teknik *failover*. Beberapa keunggulan tersebut tepat digunakan pada penelitian yang akan dilakukan.

Penelitian yang dilakukan oleh S. Mukherjee (2019), “*Benefits of AWS in Modern Cloud*”. Pada penelitian tersebut permasalahan yang dibahas yaitu tentang perlindungan data. Data merupakan sesuatu yang penting bagi setiap orang ataupun perusahaan, jika terjadi kebocoran data maka dapat mengakibatkan kerugian bagi pemilik data tersebut. Maka dari itu keamanan data merupakan hal penting yang harus dipertimbangkan ketika menyimpan data dalam *cloud storage*. Penelitian Mukherjee mengusulkan penggunaan Amazon Web Service (AWS) sebagai *cloud storage* yang mempunyai

kehandalan dalam menyimpan dan keamanan datanya. AWS menawarkan beberapa keunggulan penyimpanan *cloud* diantaranya *flexibility*, yaitu memungkinkan pengelolaan data dapat diakses darimana saja dan kapan saja dengan menggunakan perangkat seperti laptop, *smartphone*, *notebook* dll. Keunggulan lain dari penyimpanan menggunakan *cloud* yaitu *cost effectiveness* penghematan biaya tanpa membangun infrastruktur untuk menyimpan data. *Disaster recovery* merupakan kemampuan penyimpanan *cloud* untuk meminimalkan terjadinya *downtime* ketika terjadi bencana, data dilindungi di tempat yang aman dengan tetap dapat diakses oleh pengguna *cloud storage*. Hasil penelitian tersebut menunjukkan jika penggunaan AWS *cloud* mempunyai kehandalan dalam meminimalkan *downtime*, memiliki keamanan data yang baik, pengontrolan mudah, *backup* dan *recovery* yang cepat. Beberapa keunggulan AWS tersebut digunakan sebagai pertimbangan untuk penggunaan AWS *cloud*. AWS *cloud* digunakan sebagai media penyimpanan data dalam implementasi *IoT data storage*.

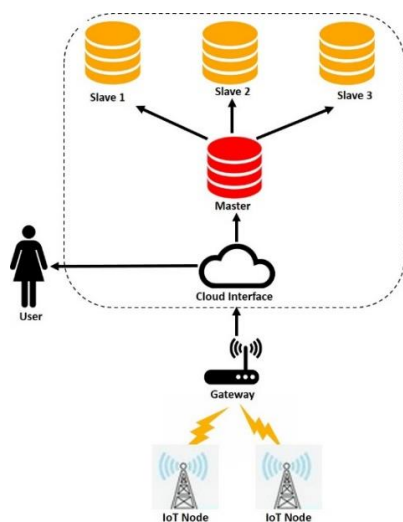
3. PERANCANGAN SISTEM

Perancangan sistem dibuat bertujuan untuk menjabarkan serta menjelaskan alur kerja sistem yang akan dibangun. Perancangan terdiri dari perancangan komponen utama yaitu konfigurasi AWS dan MySQL cluster. Selanjutnya merancang komponen pendukung yang digunakan meliputi ARC, Web flask, serta skenario pengujian menggunakan Jmeter.

3.1. Konfigurasi Amazon Web Services

Perancangan sistem dimulai dengan konfigurasi pada Amazon Web Services (AWS). Sistem dirancang dengan menggunakan beberapa AWS sebagai multinodenya. Dalam penelitian ini pengguna mengakses *cloud interface*, dimana pengguna tersebut dapat melakukan berbagai hal seperti mengunggah, mengunduh, bahkan mengubah atau mengedit data. Sedangkan *IoT node* berperan sebagai sensor yang berfungsi untuk mengambil data dan membangun koneksi ketika ada pengguna yang akan terhubung dengan *cloud interface*. *IoT node* akan membangun koneksi menuju ke *gateway*. Kemudian *gateway* menangkap koneksi tersebut lalu diteruskan ke *cloud interface*, yang selanjutnya data yang telah diubah dikirimkan ke master database. Di

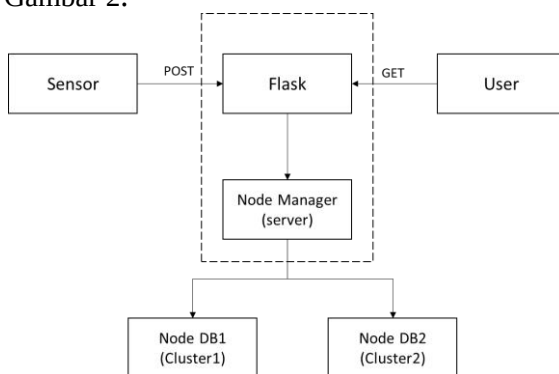
dalam master *database* data itu diolah, dipilih, dan disortir berdasarkan jenis datanya. Setelah itu data yang telah disortir tadi direplikasi ke *database Slave 1*, *Slave 2*, dan *Slave 3*. Penggunaan 3 *database slave* bertujuan untuk back up data dan berjaga-jaga jika master dan salah satu *slave database* dalam keadaan down. Fungsi replikasi adalah untuk berjaga-jaga apabila master *database* mengalami kerusakan atau down, namun data tetap bisa diakses dari *Slave 1*, *Slave 2*, ataupun *Slave 3*. Sehingga sistem dapat selalu menyediakan layanan yang bersifat *high availability*. Arsitektur *IoT data storage* dapat dilihat pada Gambar 1.



Gambar 1. Arsitektur *IoT Data Storage* dengan AWS

3.2. Perancangan MySQL cluster

Perancangan MySQL *cluster* diawali dengan konfigurasi *server* sebagai *node manager*, kemudian dua server lainnya berperan sebagai *data node*. Berikut alur kerja secara umum pada MySQL *cluster* disajikan pada Gambar 2.



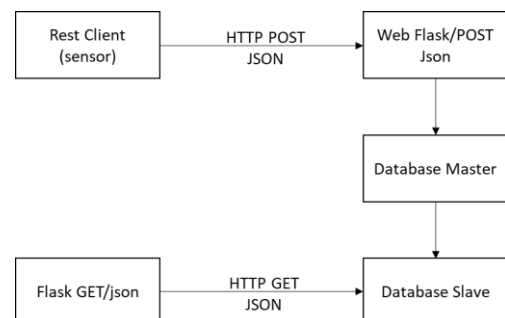
Gambar 2. Alur kerja MySQL *cluster*

Pada Gambar 2 MySQL *cluster* terdiri dari *Node Manager* sebagai *server* yang

menentukan data akan dikirimkan menuju *node DB1* atau *node DB2* sebagai *cluster*. MySQL *cluster* memiliki teknik *failover*, yaitu mode operasional cadangan yang bertujuan untuk membuat sistem lebih toleran terhadap kesalahan baik ketika mengalami kegagalan/*downtime*. Teknik tersebut dilakukan dengan cara memindahkan jalur secara otomatis ke jalur koneksi lainnya yang telah siap siaga sebagai *backup*. Seperti pada Gambar 2 ketika *node DB1* mengalami kegagalan maka data secara otomatis akan dikirimkan ke jalur *node* lainnya yaitu *Node DB2*. Selain teknik *failover* terdapat juga teknik *load balancing* yaitu pembagian beban agar salah satu server tidak mengalami *overload* ketika banyak pengguna yang mengakses. Pada gambar diatas ketika *node DB1* telah memiliki jumlah pengguna yang sudah melebihi batas maksimalnya, maka sebagian user akan ditangani oleh *node DB2*.

3.3. Perancangan ARC sensor

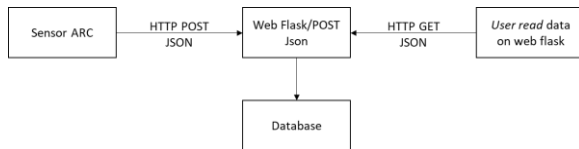
Advanced Rest Client (ARC) digunakan sebagai sensor pengganti untuk suhu dan kelembaban. Sensor tersebut memberikan data *input* untuk mengisi data dalam *database*. Berikut gambaran proses kerja sensor dalam sistem yang akan dibangun dapat dilihat pada Gambar 3.



Gambar 3. Alur kerja ARC sensor

3.4. Perancangan Web Flask

Web flask merupakan *web* yang digunakan untuk menampilkan data yang diterima oleh *database* dari sensor ARC. *Web flask* digunakan sebagai *user interface* ketika ada data masuk. Data masuk dari sensor ARC ditampilkan pada *web flask* dalam bentuk data json. Berikut alur penampilan data pada *web flask* yang dapat dilihat pada Gambar 4.

Gambar 4. Alur *web flask* menampilkan informasi

3.5. Perancangan Struktur Tabel

Perancangan struktur tabel dalam *database* digunakan untuk menentukan nama, tipe data yang digunakan dan ukuran data yang bisa dimasukan. Pada penelitian yang akan dilakukan menggunakan satu tabel, yaitu tabel sensor. Struktur tabel *database* ditunjukkan pada Tabel 1.

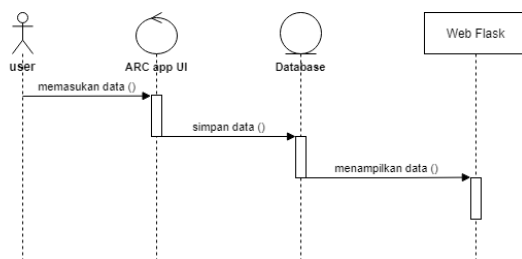
Tabel 1. Struktur tabel *database*

Nama Field	Tipe Data	Ukuran Data
Id	Int	Auto
Sensor	Varchar	20
Kelembaban	Int	10
Suhu	Int	10
Waktu	Timestamp	Auto

3.6. Perancangan Alur Komunikasi Sistem

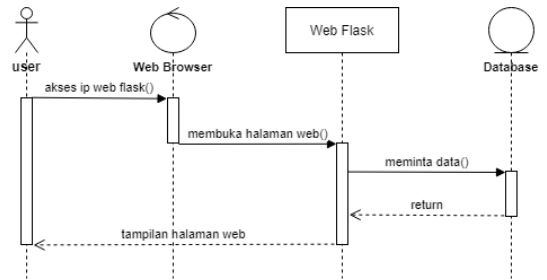
Perancangan alur komunikasi sistem digunakan untuk menentukan urutan komunikasi data dari awal perangkat sampai ke *database*. Alur komunikasi sistem disajikan dalam bentuk *sequence diagram* yang ditunjukkan pada gambar dibawah:

1. Memasukan data

Gambar 4. *Sequence diagram* memasukan data

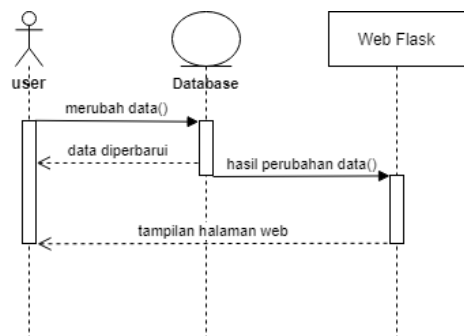
Pada Gambar 4 menunjukkan alur memasukan data kedalam *database* yang dimulai dari *user* membuka aplikasi *ARC* yang digunakan untuk memasukan data sensor yaitu nilai kelembaban dan suhu.

2. Membaca data

Gambar 5. *Sequence diagram* membaca data

Pada Gambar 5 menunjukkan alur membaca data yang diawali dengan *user* memasukan alamat ip *web flask* pada *web browser*. Ketika proses meminta halaman *web flask* terdapat proses lainnya yaitu *web flask* meminta data ke *database*. *Database* memberikan data kepada *web flask* yang selanjutnya *web flask* menampilkan data yang ada pada halaman *web* yang diminta oleh *user*.

3. Merubah data

Gambar 6. *Sequence diagram* merubah data

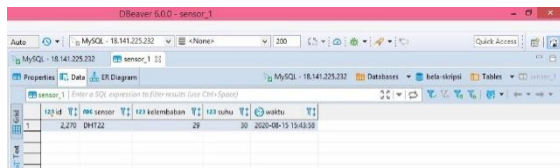
Pada Gambar 6 menunjukkan alur proses perubahan data yang dilakukan oleh *user*. *User* mengakses *database* dan melakukan perubahan data menggunakan aplikasi *Database Management System (DBMS)*. *User* menyimpan perubahan data dalam *database*. *User* mengakses alamat ip *web flask* dan *web flask* menampilkan halaman *web* dengan data yang sudah diperbarui oleh *user*.

4. PENGUJIAN DAN HASIL

4.1. Pengujian Fungsional

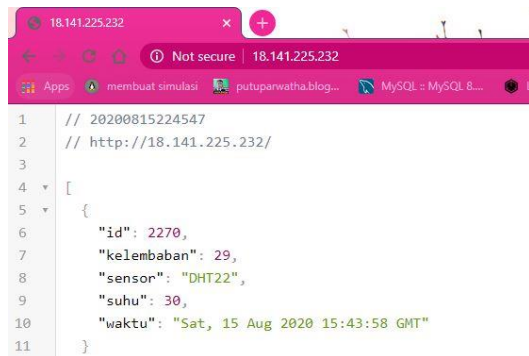
Pengujian fungsional bertujuan untuk mengetahui jika implementasi *IoT data storage* menggunakan *MySQL cluster* dapat berjalan sesuai tujuan dari penelitian. Pengujian fungsional dikatakan berhasil jika sistem dapat berjalan sesuai fungsinya yang mampu menyimpan, menghapus, mengedit data pada

cloud storage.



Gambar 7. Data tersimpan dalam database MySQL cluster

Gambar 7 menunjukkan data yang sudah tersimpan dalam database. Pada sisi *client* digunakan web flask untuk melihat data yang tersimpan pada database MySQL cluster. Web flask berfungsi sebagai *user interface* data yang terdapat pada database. Gambar 8 di bawah ini menampilkan *user interface* dari web yang diakses oleh *client*.



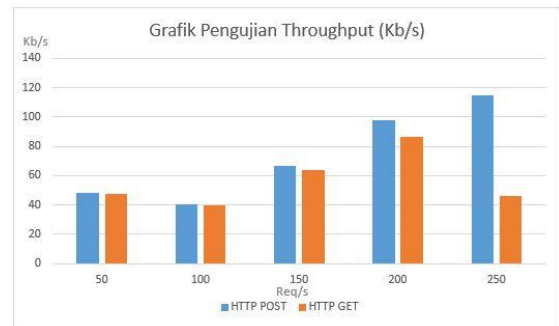
Gambar 8. Tampilan halaman web flask

4.2. Pengujian dan Hasil Kinerja dengan Parameter Throughput

Pengujian *throughput* dilakukan untuk mengetahui kinerja dari database terdistribusi dengan MySQL cluster untuk IoT data storage. Prosedur pengujian dilakukan sesuai dengan menggunakan variasi banyaknya *user* atau *Request Per Second (RPS)*. Variasi *rps* mulai dari 50 – 250 rps. Hasil pengujian didapatkan dari lima kali pengujian untuk mendapatkan hasil yang akurat dengan diambil nilai rata-rata untuk parameter *throughput*. Hasil pengujian *throughput* ditunjukkan pada Tabel 2.

Tabel 2. Hasil pengujian *throughput*

No	Request per Second	Nilai <i>throughput</i> (Kb/s)	
		HTTP POST	HTTP GET
1	50	48,52	47,26
2	100	40,76	40,16
3	150	66,44	63,54
4	200	97,74	86,20
5	250	115,10	45,86



Gambar 9. Grafik hasil pengujian *throughput*

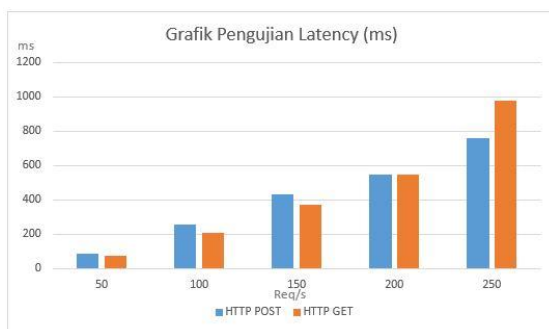
Pada Gambar 9 menampilkan hasil pengujian *throughput* untuk masing-masing *request per second (rps)*. *Throughput* dibagi menjadi dua yaitu *throughput* ketika proses GET dan POST data. Ketika *request* 50 – 150 rps nilai *throughput*-nya hampir sama untuk GET dan POST. Saat *request* mencapai 200 – 250 rps nilai *throughput* antara GET dan POST memiliki perbedaan yang cukup signifikan. Pada 200 rps GET memiliki nilai *throughput* 86,20 Kb/s sedangkan POST memiliki nilai *throughput* 97,74 Kb/s yang berarti memiliki perbedaan sebesar 11,54 Kb/s. Pada 250 rps GET memiliki nilai *throughput* yang kecil yaitu 45,86 Kb/s sedangkan pada POST memiliki nilai sebesar 115,10 Kb/s dengan perbedaan sebesar 69,24 Kb/s. Perbedaan tersebut dapat terjadi dikarenakan semakin tinggi *request* maka beban yang diterima server semakin berat sehingga nilai *throughput* mengalami penurunan meskipun tidak sampai terjadi *package drop*. Selain itu terdapat faktor lain yang mempengaruhi hasil dari penelitian ini, yaitu koneksi jaringan internet. Jaringan internet yang digunakan cenderung memiliki kecepatan yang tidak stabil sehingga setiap percobaan memiliki hasil yang berbeda-beda. Seperti yang terjadi pada GET 250 rps dimana menunjukkan nilai yang kecil dibandingkan POST 250 rps.

4.3. Pengujian dan Hasil Kinerja dengan Parameter Latency

Pengujian *latency* dilakukan untuk mengetahui waktu yang dibutuhkan ketika proses pengiriman data. Hasil pengujian *latency* untuk implementasi IoT data storage pada MySQL cluster dilakukan dengan variasi nilai 50 – 250 rps. Setiap variasi rps dilakukan pengujian sebanyak lima kali. Hasil pengujian *latency* disajikan pada tabel 3.

Tabel 3. Hasil pengujian *latency*

No	Req/s	Nilai <i>latency</i> (ms)	
		HTTP POST	HTTP GET
1	50	89,29	73,64
2	100	258,87	208,29
3	150	432,29	369,63
4	200	546,16	545,90
5	250	760,58	981,06



Gambar 10. Grafik hasil pengujian *latency*

Dari Gambar 10 menunjukkan hasil pengujian *latency* untuk GET dan POST dengan nilai *request* 50 – 250 rps. Dari grafik tersebut menunjukkan jika semakin tinggi jumlah *request* maka nilai *latency* semakin naik yang berarti semakin lama waktu yang dibutuhkan untuk melayani *request*. Pada *request* 50 rps memiliki nilai *latency* terendah hal tersebut dikarenakan jumlah *request* yang dilayani tidak terlalu banyak. Ketika *request* 250 rps memiliki nilai *latency* tertinggi dalam grafik tersebut yang disebabkan karena tingginya *request* yang dilayani oleh *server* sehingga memerlukan waktu yang lebih lama. Nilai *latency* memang terus meningkat seiring bertambahnya jumlah *request* hal tersebut masih dapat ditoleransi karena *latency* masih dalam *milli second* dan tidak sampai terjadi *request time out*.

5. KESIMPULAN

Kesimpulan yang didapat dari implementasi IoT *data storage* dengan MySQL *cluster*, yaitu:

1. Implementasi IoT *data storage* dengan MySQL *cluster* dilakukan dengan menggunakan Amazon Web Service sebagai pengelola server yang terhubung dengan database MySQL *cluster*. Sensor IoT yang digunakan yaitu suhu dan kelembaban yang digantikan dengan aplikasi *Advanced Rest Client*. Komponen lain yang digunakan yaitu *web flask* untuk menampilkan data pada database ke halaman web yang diakses oleh *client*.

Sistem yang dibangun dapat menjalankan fungsinya dengan baik sesuai dengan tujuan pada penelitian.

2. Pengujian kinerja dibagi menjadi dua parameter, yaitu *throughput* dan *latency*. Nilai *throughput* tertinggi yang didapat yaitu 115,10 Kb/s pada POST 250 rps. Ketika terjadi peningkatan rps maka nilai *throughput* juga meningkat, tetapi ketika kecepatan jaringan internet tidak stabil dapat mempengaruhi besar kecilnya nilai *throughput* seperti yang terjadi pada GET 250 rps. Sedangkan *latency* tercepat yaitu 89,29 ms pada 50 rps. Semakin tinggi *request* maka nilai *latency* semakin tinggi, hal tersebut dikarenakan banyaknya *request* yang dilayani *server* maka semakin lama waktu yang diperlukan.

6. DAFTAR PUSTAKA

- Ali, H. & Badaway, M. (2015). *Internet of Things (IoT): Definitions, Challenges and Recent Research Directions*. *International Journal of Computer Applications*.
- Andhikha, I. (2018). Implementasi dan Analisis Kinerja MySQL Cluster Menggunakan Metode Load Balancing. *Jurnal Manajemen Informatika*.
- Dhaka, P. & Johari, R. (2016). *Big data application: Study and archival of mental health data, using MongoDB*. *International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT)*.
- Ezechiel, K., Kant, S., & Agarwal, R. (2019). A Systematic Review On Distributed Databases Systems and Their Techniques. *Journal of Theoretical and Applied Information Technology*
- Gani, T., Arafat A., & Melinda. (2016). Analisis Kinerja MySQL Cluster Menggunakan Metode Load Balancing.
- Guoqiang, S., Chao, Z., Yanming, C., & Yanxu, Z. (2013). *Design and Implementation of a Smart IoT Gateway*. 10.1109/GreenCom-iThings-CPSCoM.2013.130.
- Kamalakaran, T., Senthil, K., Shanthi, C., & Radhakrishnan, D. (2019). *Study on Cloud Storage and its Issues in Cloud Computing*. *International Journal of Management, Technology and*

- Engineering*.
- Kumar, S., Tiwari, P., & Zymbler, M. (2019). *Internet of Things is a revolutionary approach for future technology enhancement: a review*.
- Mekki, K ., Bajic, E., Chaxel, F., & Meyer, F. (2019). *Concept and Hardware Considerations for Product-Service System Achievement in Internet of Things*.
- Mukherjee, S. (2019). *Benefits of AWS in Modern Cloud*.
- Permatasari, D., Ardani, M., Ma'ulfa A., & Ilhami, N. (2020). Pengujian Aplikasi Menggunakan Metode *Load Testing* dengan Apache Jmeter pada Sistem Informasi Pertanian. *Jurnal Sistem dan Teknologi Informasi*.
- Prabowo, A. & Soemantri, M. (2011). Perancangan MySql Cluster untuk Mengatasi Kegagalan Sistem Basis Data pada Sisi Server. *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer*.
- Pratama, O. (2018). Pengembangan Perangkat Lunak IoT Cloud Platform Berbasis Protokol Komunikasi HTTP. *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer*.
- Prayoga, M. (2018). Pengertian dan Komponen SQL.
- Pukdesree, S., Lacharaj, W., & Sirisang, P. (2010). *Performance evaluation of distributed database on PC cluster computers*. *WSEAS Transactions on Computers*.
- Swara, G & Pebriadi, Y. (2016). Rekayasa Perangkat Lunak Pemesanan Tiket Bioskop Berbasis Web. *Jurnal TEKNOIF*.
- Warman, I & Ramdaniansyah, R. (2018). Analisis Perbandingan Kinerja *Query Database Management System (DBMS)* antara MySQL 5.7.6 dan MariaDB 10.1.
- Wulandari, R. (2016). Analisis QoS (Quality of Service) pada Jaringan Internet (Studi Kasus: UPT Loka Uji Teknik Penambangan Jampang Kulon – LIPI). *Jurnal Teknik Informatika dan Sistem Informasi* e-ISSN : 2443-2229 Volume 2 Nomor 2 Agustus 2016.
- Zhang, Q., Cheng, L., & Boutaba, R. (2010). *Cloud computing: state-of-the-art and research challenges*.